

TECHNICAL REQUIREMENT DOCUMENT

Multi-Tenant Property Management SaaS Platform

System Core Architectural Blueprint
Version 2.1.0 · Baseline-Locked

Document Title	TRD — Multi-Tenant Property Management SaaS Platform
Lead Systems Architect	Befiker Gezahegn Hailemichael
Target Environment	Java 21 (Eclipse Temurin OpenJDK) / Spring Boot / PostgreSQL
Document Version	2.1.0
Date	June 2026
Classification	Internal — Confidential

Table of Contents

1. Framework & Application Runtime Environment
2. Multi-Schema Database Topology (PostgreSQL)
3. Multi-Tenant Isolation Middleware Contract
4. Financial Safety & Strict Relational Constraints
5. Asynchronous Audit Ledger Architecture
6. Automated Verification Matrix (Testing Architecture)

1. Framework & Application Runtime Environment

This section formally specifies the runtime, framework, and connection-handling configuration that the entire backend system is built upon. Every subsequent architectural decision in this document — the schema topology, the isolation middleware, the financial constraints, and the audit ledger — assumes this runtime baseline is in place exactly as specified here. No deviation from this baseline configuration is permitted without a corresponding revision to this document.

1.1 Base Stack Configuration

The application layer must be constructed strictly using Java 21, distributed via the Eclipse Temurin OpenJDK build, to leverage virtual threads under Project Loom and the modern language features introduced across the Java 17 through 21 release train. Eclipse Temurin is selected specifically because it is a vendor-neutral, TCK-certified OpenJDK distribution that carries no commercial licensing obligation, in contrast to Oracle's JDK distribution under its post-Java-17 licensing terms.

The framework runtime is Spring Boot 3.x, selected for its production-grade auto-configuration model, its native compatibility with the Java 21 baseline, and its mature ecosystem of starter dependencies that map directly onto every architectural requirement in this document. The following core dependency ecosystem constitutes the mandatory and complete set of framework-level dependencies for the Phase 1 backend:

- **Spring Data JPA (Hibernate 6.x):** Provides the object-relational mapping layer and all database communication for every entity defined in Section 2. Hibernate 6.x is required specifically for its native support of the Spring Data JPA filter mechanism used in the tenant isolation strategy detailed in Section 3.2.
- **Spring Security:** Operated in a fully stateless configuration using JSON Web Tokens for authentication, with method-level access control enforced through the `@PreAuthorize` annotation family. No server-side session state is held anywhere in the authentication or authorization path.
- **Liquibase Core:** The exclusive mechanism for managing version-controlled database migrations across every environment and every runtime update. No schema change of any kind — including the constraints specified in Section 4 — may be applied to any environment outside of a committed Liquibase changeset.
- **Spring Batch Core:** Provides the job execution framework for the complex, long-running background invoicing operations that populate the `public.invoices` table defined in Section 2.2, and whose framework-level metadata is isolated into the `spring_batch` schema defined in Section 2.1.

1.2 Embedded Container & Connection Pool Specs

The platform embeds a production-hardened Tomcat container rather than relying on an external, separately deployed application server. This embedded model is consistent with the single deployable artifact philosophy of Spring Boot and simplifies the container orchestration surface to a single process per application instance.

Performance tuning parameters for both the embedded Tomcat container and the HikariCP connection pool are explicitly fixed at the following values, calibrated for a highly concurrent multi-tenant SaaS runtime environment under realistic Phase 1 production load assumptions:

`application.yml` — Embedded Container & Connection Pool Configuration

```

server:
  tomcat:
    threads:
      max: 200
      min-spare: 20
    accept-count: 100
    max-connections: 8192

spring:
  datasource:
    type: com.zaxxer.hikari.HikariDataSource
    hikari:
      maximum-pool-size: 25
      minimum-idle: 5
      idle-timeout: 300000
      max-lifetime: 1800000
      connection-timeout: 30000
      pool-name: EnterprisePropertySaaS HikariCP

```

1.2.1 Tomcat Thread Pool Parameters

Parameter	Value & Purpose
threads.max: 200	The upper bound on concurrent request-handling worker threads. Calibrated to accommodate burst traffic from multiple simultaneously active tenant organizations without resource exhaustion on a standard MVP-scale compute allocation.
threads.min-spare: 20	The minimum number of idle worker threads retained at all times, ensuring the container can respond immediately to a traffic burst without incurring the latency cost of cold thread creation.
accept-count: 100	The size of the request queue once all 200 worker threads are saturated. Requests beyond this queue depth are rejected at the TCP layer rather than accepted and left to wait indefinitely.
max-connections: 8192	The maximum number of simultaneous TCP connections the container will accept, set well above the thread pool ceiling to accommodate connections that are open but not currently being actively processed, such as idle keep-alive connections from client applications.

1.2.2 HikariCP Connection Pool Parameters

Parameter	Value & Purpose
maximum-pool-size: 25	The hard ceiling on simultaneous database connections held by a single application instance. Set deliberately below the Tomcat thread ceiling, since not every web request requires a database connection for its full duration, and PostgreSQL's own connection ceiling must be respected across however many application instances are running concurrently.
minimum-idle: 5	The floor of idle connections HikariCP keeps warm at all times, avoiding the latency cost of establishing a fresh database connection on every traffic uptick.

idle-timeout: 300000	Idle connections beyond the minimum-idle floor are closed after five minutes (300,000 milliseconds) of inactivity, releasing database-side resources during quiet periods.
max-lifetime: 1800000	Every connection, regardless of activity, is retired and replaced after thirty minutes (1,800,000 milliseconds), preventing the accumulation of long-lived connections that may be holding stale routing or replication state in a managed PostgreSQL environment.
connection-timeout: 30000	The maximum time, in milliseconds, a request will wait for a connection to become available from the pool before failing with an explicit connection-acquisition exception, rather than hanging indefinitely.
pool-name: EnterprisePropertySaaS HikariCP	An explicit, human-readable pool identifier surfaced in JMX metrics and structured logs, allowing connection pool behavior to be unambiguously identified in a production environment running multiple distinct connection pools across different subsystems.

2. Multi-Schema Database Topology (PostgreSQL)

To achieve absolute database safety and structural isolation, the enterprise data architecture uses a dual-schema design within a single primary PostgreSQL instance. This section documents that topology exhaustively, table by table, establishing the precise data dictionary that the Liquibase changesets referenced throughout this document, and authored in the subsequent database layer setup phase, must conform to without deviation.

Dual-Schema Topology Overview

PostgreSQL Primary Instance			
Business Domain	public schema (Isolated Batch Engine Meta)	spring_batch schema	(Multi-Tenant)
- organizations	- BATCH_JOB_INSTANCE	- properties (with tenant_id)	
- BATCH_JOB_EXECUTION	- property_structures	-	
BATCH_STEP_EXECUTION	- units (with tenant_id)	-	
BATCH_JOB_EXECUTION_CONTEXT	- leases (with tenant_id)		-
invoices (with tenant_id)		- system_audit_logs	

2.1 Schema Segregation Rationale

public Schema

Contains all core tenant business entities. Every multi-tenant data row is mapped to a strict structural organization identity boundary by way of its `tenant_id` column, with the sole architectural exception of the `organizations` table itself, which serves as the root identity boundary and therefore carries no `tenant_id` of its own.

spring_batch Schema

Houses framework metadata tables utilized by the Spring Batch automated generation engine, specifically the `BATCH_JOB_INSTANCE`, `BATCH_JOB_EXECUTION`, `BATCH_STEP_EXECUTION`, and `BATCH_JOB_EXECUTION_CONTEXT` tables that Spring Batch itself creates and manages internally.

Architectural Rationale: Isolating these framework engine tables completely ensures that system background batch processing does not trigger or conflict with application row-level security filters keyed on `tenant_id`, which would otherwise cause batch metadata lookup statements to fail. The Spring Batch framework's internal bookkeeping tables have no notion of `tenant_id` whatsoever — a single batch job execution record describes a job run that, by its very nature, processes leases and invoices belonging to many different tenant organizations within one execution. Attempting to apply a tenant-scoped row-level security policy to these framework tables would either require Spring Batch to be modified to set a tenant context it has no business knowing about, or would silently and incorrectly filter the batch framework's own operational visibility into its own job history. Complete schema-level separation, governed by two distinct PostgreSQL database roles, removes this conflict entirely at the structural level rather than attempting to manage it through application-level discipline.

2.2 Core Data Dictionary

The following data dictionary defines every column, data type, nullability rule, key relationship, and indexing requirement for each table in the public schema. UUID is used explicitly for every primary and foreign key across the business domain, with the sole exception of the `system_audit_logs` table's own primary key, which is justified individually in that table's subsection below.

public.organizations

Defines the root enterprise system tenant. It contains no `tenant_id` because it serves as the parent isolation identity boundary against which every other table's `tenant_id` ultimately resolves.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
<code>id</code>	UUID	No	PRIMARY KEY	Unique organization token.
<code>name</code>	VARCHAR(255)	No	—	UNIQUE constraint per database.
<code>created_at</code>	TIMESTAMP	No	—	Default CURRENT_TIMESTAMP.
<code>updated_at</code>	TIMESTAMP	No	—	Default CURRENT_TIMESTAMP.

public.properties

Captures unique real estate physical asset complexes registered to an organization.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
<code>id</code>	UUID	No	PRIMARY KEY	Unique asset token.
<code>tenant_id</code>	UUID	No	FOREIGN KEY	References public.organizations(id), INDEXED.
<code>name</code>	VARCHAR(255)	No	—	—
<code>address</code>	TEXT	No	—	—
<code>location_city</code>	VARCHAR(100)	No	—	B-Tree INDEXED for fast location filtering.
<code>created_at</code>	TIMESTAMP	No	—	—

public.property_structures

Manages logical nesting layers (Blocks, Floors, Wings) within a specific property asset.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
<code>id</code>	UUID	No	PRIMARY KEY	Unique structure node token.
<code>tenant_id</code>	UUID	No	FOREIGN KEY	References public.organizations(id), INDEXED.
<code>property_id</code>	UUID	No	FOREIGN KEY	References public.properties(id), INDEXED.
<code>parent_id</code>	UUID	Yes	FOREIGN KEY	References public.property_structures(id).
<code>node_type</code>	VARCHAR(50)	No	—	E.g., BLOCK, FLOOR, WING.
<code>name</code>	VARCHAR(100)	No	—	—

public.units

Represents individual rentable operations slots (offices, suites, retail shops).

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
id	UUID	No	PRIMARY KEY	Unique unit token.
tenant_id	UUID	No	FOREIGN KEY	References public.organizations(id), INDEXED.
property_id	UUID	No	FOREIGN KEY	References public.properties(id), INDEXED.
structure_id	UUID	Yes	FOREIGN KEY	References public.property_structures(id).
unit_number	VARCHAR(50)	No	—	Combined INDEXED with property_id.
status	VARCHAR(30)	No	—	VACANT, OCCUPIED, MAINTENANCE.
baseline_price	NUMERIC(15, 2)	No	—	Checked scale enforcement.
currency_code	VARCHAR(3)	No	—	Default ETB.

public.leases

Establishes formal tenant contractual bounds mapped against individual unit inventory spaces.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
id	UUID	No	PRIMARY KEY	Unique lease contract token.
tenant_id	UUID	No	FOREIGN KEY	References public.organizations(id), INDEXED.
unit_id	UUID	No	FOREIGN KEY	References public.units(id), INDEXED.
status	VARCHAR(30)	No	—	DRAFT, PENDING, ACTIVE, EXPIRED.
start_date	DATE	No	—	—
end_date	DATE	No	—	—

public.invoices

Stores financial statements generated automatically by periodic billing execution scripts.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
id	UUID	No	PRIMARY KEY	Unique invoice transaction token.
tenant_id	UUID	No	FOREIGN KEY	References public.organizations(id), INDEXED.
lease_id	UUID	No	FOREIGN KEY	References public.leases(id), INDEXED.
billing_period	VARCHAR(7)	No	—	Format: YYYY-MM.
amount_due	NUMERIC(15, 2)	No	—	—
status	VARCHAR(30)	No	—	UNPAID, PARTIALLY_PAID, PAID.

public.system_audit_logs

Chronological central ledger table for change tracking, written exclusively through the asynchronous architecture defined in Section 5.

Column Name	Data Type	Nullable	PK / FK	Constraints / Indexes
id	BIGSERIAL	No	PRIMARY KEY	Incremental identity index.
tenant_id	UUID	No	—	B-Tree INDEXED.
actor_id	UUID	No	—	—
entity_type	VARCHAR(100)	No	—	E.g., LEASING, INVENTORY.
entity_id	UUID	No	—	INDEXED.
action_type	VARCHAR(50)	No	—	CREATE, UPDATE, DELETE.
old_value	JSONB	Yes	—	—
new_value	JSONB	Yes	—	—
timestamp	TIMESTAMP	No	—	Default CURRENT_TIMESTAMP.

Architectural Rationale: The `system_audit_logs` table is the single deliberate exception to the UUID-everywhere convention used across the rest of this data dictionary. Its primary key is a `BIGSERIAL` rather than a `UUID` specifically because this table is append-only, exceptionally high in write volume relative to every other table in the schema, and is almost universally queried in strict chronological order to reconstruct an event timeline for compliance or operational forensics. A `BIGSERIAL` guarantees monotonic insertion ordering without requiring a secondary sort column, and its index footprint at scale is meaningfully smaller than the equivalent `UUID`-keyed index would be.

3. Multi-Tenant Isolation Middleware Contract

To fulfill the zero data crosstalk protection standard mandated by the product definition rules, data operations use an automated logical multi-tenant interception pipeline. This section specifies, with full implementation-level precision, exactly how that pipeline harvests tenant context from an inbound request and propagates it, without exception, into every database operation performed on behalf of that request.

3.1 Token Context Harvesting Sequence

The following sequence executes identically for every single inbound HTTP request reaching the application, with no exceptions carved out for any endpoint that touches tenant-scoped data.

1. Every client HTTP call hitting the server boundary requires a valid Bearer JWT inside the Authorization request header context. Requests arriving without this header, or with a malformed header, are rejected before any further processing occurs.
2. A custom stateless JwtAuthenticationFilter, extending Spring Security's OncePerRequestFilter, intercepts the request at the earliest point in the security filter chain, ensuring no downstream filter or controller logic executes before tenant context has been established.
3. The filter parses the token signature using the shared cryptographically signed key, extracts the tenant identifier token claim (org_id), and binds it securely to an explicit execution container thread wrapper referred to throughout this document as TenantContext.

TenantContext.java – Thread-Local Tenant Identity Wrapper

```
public class TenantContext {
    private static final ThreadLocal<UUID> currentTenant = new ThreadLocal<>();

    public static void setCurrentTenantId(UUID tenantId) {
        currentTenant.set(tenantId);
    }

    public static UUID getCurrentTenantId() {
        return currentTenant.get();
    }

    public static void clear() {
        currentTenant.remove();
    }
}
```

Architectural Rationale: A ThreadLocal-backed context wrapper is used, rather than passing the tenant identifier explicitly as a parameter through every layer of the application, because it allows the tenant boundary to be transparently available to any code executing on the request-handling thread — including the repository-layer interception described in Section 3.2 — without requiring every service and repository method signature in the codebase to be rewritten to accept an additional parameter. The TenantContext.clear() call is mandatory at the end of every request's filter chain, including on the exception path, to guarantee that thread pool reuse under Tomcat's worker thread model can never leak one request's tenant identity into a subsequently handled, unrelated request on the same reused thread.

3.2 Automated Repository Scoping

To remove the risk of manual query coding oversights, all relational retrievals use Hibernate dynamic query injection rather than relying on every individual repository method author to remember to add an explicit WHERE clause filtering on `tenant_id`. Every domain entity that is tenant-dependent must explicitly define the `@FilterDef` and `@Filter` annotations shown below, inherited from a single shared mapped superclass.

TenantEntity.java – Shared Mapped Superclass with Hibernate Filter Definition

```
@MappedSuperclass
@FilterDef(name = "tenantFilter", parameters = @ParamDef(name = "tenantId", type =
UUID.class))
@Filter(name = "tenantFilter", condition = "tenant_id = :tenantId")
@Data
public abstract class TenantEntity {
    @Column(name = "tenant_id", nullable = false, updatable = false)
    private UUID tenantId;
}
```

A dedicated Spring Aspect Oriented Programming component intercepts Spring Data JPA repository execution entry points to bind this filter's parameters safely before any query against a tenant-scoped repository executes.

TenantFilterAspect.java – AOP Interceptor Binding the Hibernate Filter Before Repository Execution

```
@Aspect
@Component
public class TenantFilterAspect {
    @PersistenceContext
    private EntityManager entityManager;

    @Before("execution(* com.enterprise.repository..*.*(..))")
    public void activateTenantFilter() {
        Session session = entityManager.unwrap(Session.class);
        UUID currentTenantId = TenantContext.getCurrentTenantId();
        if (currentTenantId != null) {
            Filter filter = session.enableFilter("tenantFilter");
            filter.setParameter("tenantId", currentTenantId);
        }
    }
}
```

Architectural Rationale: Binding the Hibernate filter through an AOP advice executing immediately before every repository method invocation, rather than relying on each repository method to independently apply a `tenant_id` predicate, converts tenant isolation from a convention that an individual developer could forget into a structural guarantee that applies uniformly across the entire `com.enterprise.repository` package by virtue of the pointcut expression alone. A newly added repository method, written by any engineer at any point in the project's life, inherits this protection automatically and cannot be coded in a way that accidentally omits it, because the filtering occurs outside of and prior to the method's own body.

4. Financial Safety & Strict Relational Constraints

This section codifies, with exact and unambiguous specification, the numerical precision rules and the two structural database constraints that the platform's financial and lease integrity guarantees depend upon. These constraints are not application-layer conventions; they are enforced by the PostgreSQL database engine itself and are authored as committed Liquibase changesets, applied identically and automatically across every environment the platform is ever deployed to.

4.1 Numerical Precision Configuration

All financial value abstractions inside the application runtime logic layer must exclusively utilize `java.math.BigDecimal` structures. The native Java primitive types `float` and `double` are permanently and unconditionally banned from representing any monetary value anywhere in the codebase, in any layer, without exception.

Intercepting calculations must restrict rounding precision deviations strictly to Banker's Rounding constraints — formally, `RoundingMode.HALF_EVEN` — applied consistently at every point a monetary `BigDecimal` value is scaled or rounded.

Example – BigDecimal Arithmetic with Banker's Rounding

```
BigDecimal monthlyRent = new BigDecimal("45000.00");
BigDecimal calculatedTax = monthlyRent.multiply(new BigDecimal("0.15"))
    .setScale(2, RoundingMode.HALF_EVEN);
```

Architectural Rationale: `RoundingMode.HALF_EVEN`, commonly referred to as Banker's Rounding, is mandated over the more intuitive `RoundingMode.HALF_UP` specifically because `HALF_UP` introduces a small but systematic upward bias when rounding values that fall exactly halfway between two representable amounts, and this bias compounds measurably across the very large number of repeated billing calculations the platform will perform over its operational lifetime. `HALF_EVEN` rounds such halfway values to whichever neighboring even digit is nearest, which is statistically unbiased over a large population of roundings and is the rounding convention financial systems standardize on for exactly this reason. Every monetary column in the schema is correspondingly typed as `NUMERIC(15,2)` at the PostgreSQL layer, ensuring the database's own storage precision matches the scale the application layer commits to.

4.2 Temporal Lease Isolation Constraint (PostgreSQL Liquibase Spec)

To enforce the validation rule that prevents a specific unit inventory node from containing more than one `ACTIVE` lease context concurrently, database management files must compile a PostgreSQL Partial Unique Index, applied through the following Liquibase changeset.

Liquibase Changeset 20260616-01 – Partial Unique Index on Active Leases

```
<changeSet id="20260616-01-add-partial-index-leases" author="architecture-team">
  <sql dbms="postgresql">
    CREATE UNIQUE INDEX idx_unique_active_lease_per_unit
    ON public.leases (unit_id)
    WHERE status = 'ACTIVE';
```

```
</sql>
</changeSet>
```

Architectural Rationale: A partial index restricted by the *WHERE* status = 'ACTIVE' filter condition, rather than an unconditional unique constraint applied across every row of the leases table regardless of status, is required because a single unit must legitimately be permitted to accumulate many historical lease records over its operational lifetime — a DRAFT lease being prepared, an EXPIRED lease from a prior tenancy, and an ACTIVE lease for the current tenancy may all coexist as rows referencing the same unit_id. An unconditional unique constraint on unit_id alone would make it structurally impossible for a unit to ever have more than one lease record in its entire history, which directly contradicts the platform's requirement to preserve historical tenancy data. Enforcing this rule at the database layer, rather than relying solely on an application-level check performed immediately before an insert statement, additionally guarantees correctness under concurrent access: two simultaneous requests attempting to activate two different leases against the same unit cannot both succeed, because PostgreSQL itself, not a race-prone application-level check-then-insert sequence, is the final and authoritative arbiter of this constraint.

4.3 Invoicing Idempotency Constraint (PostgreSQL Liquibase Spec)

To programmatically block the background batch system from generating duplicate statement profiles for a specific contract layer during an execution cycle, database configurations must enforce a hard multi-column Composite Unique Index constraint, applied through the following Liquibase changeset.

Liquibase Changeset 20260616-02 – Composite Unique Index on Lease and Billing Period

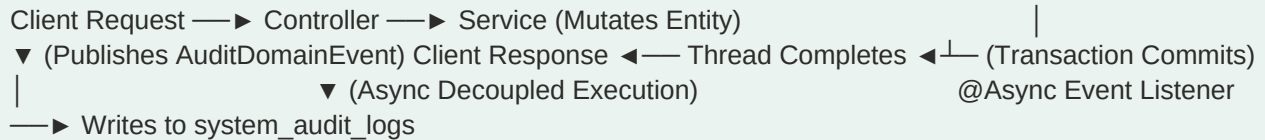
```
<changeSet id="20260616-02-add-composite-index-invoices" author="architecture-team">
  <sql dbms="postgresql">
    CREATE UNIQUE INDEX idx_prevent_duplicate_invoice_per_period
    ON public.invoices (lease_id, billing_period);
  </sql>
</changeSet>
```

Architectural Rationale: The composite unique index spanning lease_id and billing_period guarantees that the Spring Batch invoicing job, regardless of how many times it is triggered, retried after a failure, or accidentally invoked twice for the same calendar period through an operational mistake, can never insert a second invoice row describing the same lease's obligation for the same billing period. This is the structural backstop that makes the billing engine idempotent at the data layer itself — independent of whatever retry, scheduling, or error-handling logic exists at the Spring Batch job configuration level, which may itself contain bugs or be misconfigured. A unique constraint violation at the database layer is the platform's last and most trustworthy line of defense against a duplicate financial statement reaching a tenant.

5. Asynchronous Audit Ledger Architecture

To trace mutations while avoiding latency bottlenecks in operational query loops, the system relies on fully decoupled, asynchronous transaction logging. This section specifies the precise mechanics of that decoupling, from the moment a business transaction commits through to the moment the resulting audit record is durably persisted.

Asynchronous Audit Flow



5.1 Interceptor Mechanics — Asynchronous Interceptor Implementation

Application execution services publish immutable domain event markers to the Spring framework container context on state save executions. The event itself is a plain, immutable carrier of every field the eventual audit record will need — it performs no logic of its own.

AuditDomainEvent.java — Immutable Domain Event Carrier

```

@Getter
public class AuditDomainEvent extends ApplicationEvent {
    private final UUID tenantId;
    private final UUID actorId;
    private final String entityType;
    private final UUID entityId;
    private final String actionType;
    private final Object oldState;
    private final Object newState;

    public AuditDomainEvent(Object source, UUID tenantId, UUID actorId, String entityType,
        UUID entityId, String actionType, Object oldState, Object newState) {
        super(source);
        this.tenantId = tenantId;
        this.actorId = actorId;
        this.entityType = entityType;
        this.entityId = entityId;
        this.actionType = actionType;
        this.oldState = oldState;
        this.newState = newState;
    }
}

```

A specialized asynchronous engine listens for these transitions. The `@TransactionalEventListener` ensures that audit logging operations trigger only after the parent business transaction commits successfully, never before, and never if the transaction rolls back for any reason.

AsyncAuditLedgerListener.java — Transactional, Asynchronous Audit Write Handler

```

@Component
@RequiredArgsConstructor
public class AsyncAuditLedgerListener {

    private final AuditLogRepository auditLogRepository;
    private final ObjectMapper objectMapper;

    @Async("auditTaskExecutor")
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void processEntityAuditLogging(AuditDomainEvent event) {
        try {
            String jsonOldValue = event.getOldState() != null ?
objectMapper.writeValueAsString(event.getOldState()) : null;
            String jsonNewValue = event.getNewState() != null ?
objectMapper.writeValueAsString(event.getNewState()) : null;

            SystemAuditLog logRecord = SystemAuditLog.builder()
                .tenantId(event.getTenantId())
                .actorId(event.getActorId())
                .entityType(event.getEntityType())
                .entityId(event.getEntityId())
                .actionType(event.getActionType())
                .oldValue(jsonOldValue)
                .newValue(jsonNewValue)
                .build();

            auditLogRepository.save(logRecord);
        } catch (JsonProcessingException e) {
            // Internal system fault logging context isolation
            throw new CompletionException("Failed to serialize audit log tracking context",
e);
        }
    }
}

```

Architectural Rationale: Binding the listener to `TransactionPhase.AFTER_COMMIT`, rather than allowing it to fire synchronously within the same transaction as the business operation it is auditing, is essential because a transaction that publishes an event but subsequently fails and rolls back must never result in an audit record describing a state change that, from the database's authoritative point of view, never actually happened. The additional `@Async("auditTaskExecutor")` annotation then ensures that even this correctly-gated, post-commit listener invocation executes on a dedicated thread pool entirely separate from the Tomcat worker thread that is, by this point, already returning the HTTP response to the calling client — meaning the audit write's own duration, and any transient slowness in writing to `system_audit_logs`, can never be perceived by the end user as added request latency.

5.2 Serialization Protocol

State alterations are cleanly serialized into JSON structures, persisted into the `old_value` and `new_value` JSONB columns of the `system_audit_logs` table defined in Section 2.2, and logged to the central database without blocking web thread performance.

The `ObjectMapper` instance used for this serialization is the application's standard, centrally configured Jackson `ObjectMapper` bean, ensuring that the JSON representation of any given entity's state in the audit ledger is always structurally consistent with that same entity's JSON representation anywhere else in the application, such as in API responses. Any failure to serialize a given state object — for example, due to an unexpected circular reference introduced by a future entity relationship change — is treated as an internal

system fault and is raised as an explicit exception rather than being silently swallowed, ensuring that a broken audit pipeline surfaces immediately in operational monitoring rather than failing invisibly.

6. Automated Verification Matrix (Testing Architecture)

This section defines the testing specifications required to clear the Phase 4 release criteria established in the governing product requirements. Every architectural guarantee made elsewhere in this document — tenant isolation in Section 3, the relational constraints in Section 4, and the audit ledger's transactional semantics in Section 5 — must be independently and automatically verified by the test suite described here before the platform may be considered release-ready.

6.1 Unit / Integration Verification Setup

The validation harness is standardized on JUnit 5 as the test execution engine, Mockito for test-double construction and behavior verification, and AssertJ for fluent, readable assertion syntax across the entire test suite. Mocking definitions must validate complete layer segregation — a unit test exercising a service class mocks its repository dependencies rather than reaching through to a real database connection, while an integration test exercising the full request path, including the tenant isolation middleware described in Section 3, runs against a real, ephemeral PostgreSQL instance rather than a mocked persistence layer, since the isolation guarantee being tested is meaningless if the database engine itself is not genuinely present and enforcing the constraints described in Section 4.

6.2 Multi-Tenant Adversarial Assertion Specification

To guarantee code stability and satisfy release criteria, integration test patterns must explicitly simulate cross-tenant access vectors to verify that security controls prevent unauthorized data access. These tests are deliberately adversarial in framing: each test constructs a scenario in which one tenant's context attempts, whether through a coding error elsewhere in the system or a direct adversarial attempt, to read or act upon another tenant's data, and asserts that the isolation middleware described in Section 3 prevents this access from succeeding under any circumstance.

MultiTenantDataSecurityAdversarialTest.java – Cross-Tenant Isolation Adversarial Test

```
@SpringBootTest
@ActiveProfiles("test")
@ExtendWith(SpringExtension.class)
class MultiTenantDataSecurityAdversarialTest {

    @Autowired
    private PropertyService propertyService;

    @Autowired
    private PropertyRepository propertyRepository;

    @Test
    void
    shouldStrictlyIsolateQueryExecutionAndThrowExceptionWhenTenantCrossCrosstalkAttempted() {
        // Setup Isolated Business Assets for Tenant Alpha Portfolio
        UUID tenantAlphaId = UUID.randomUUID();
        Property propertyAlpha = Property.builder()
            .tenantId(tenantAlphaId)
            .name("Alpha Commercial Hub")
            .address("Bole Road")
            .locationCity("Addis Ababa")
            .build();
```

```

        propertyRepository.save(propertyAlpha);

        // Simulate AttackVector from Tenant Beta Context trying to scan Tenant Alpha
        Portfolio
        UUID tenantBetaId = UUID.randomUUID();
        TenantContext.setCurrentTenantId(tenantBetaId);

        // Execution of Target Portfolio Evaluation
        List<PropertyResponseDto> isolatedResults =
        propertyService.getAllPropertiesByTenant(tenantAlphaId);

        // Verification Rule: System must yield an empty result array or raise an execution
        fault boundary
        assertThat(isolatedResults).isEmpty();

        // Clear runtime execution thread context allocation
        TenantContext.clear();
    }
}

```

Beyond the repository-level adversarial pattern illustrated above, the full verification matrix extends this same adversarial framing to the HTTP boundary itself: explicit integration tests must programmatically attempt cross-tenant access through the REST API layer — for example, requesting a specific lease, unit, or invoice resource by its identifier while authenticated under a different tenant's JWT context — and confirm that the system intercepts these attempts and responds with either a 403 Forbidden or a 404 Not Found error response, never with the requested resource's actual data, and never with an unhandled server-side exception that might itself leak structural information about the existence or shape of the other tenant's record.

Verification Layer	Required Assertion
Repository-level adversarial test	A cross-tenant query against a tenant-scoped repository returns an empty result set, never the other tenant's records, as illustrated in the test class above.
Service-level adversarial test	A service method invoked with a resource identifier belonging to another tenant raises an explicit, typed exception rather than silently returning null or an empty optional that a calling controller might mishandle.
HTTP boundary adversarial test	An authenticated request for a specific resource identifier belonging to another tenant resolves to HTTP 403 Forbidden or HTTP 404 Not Found, confirmed across every REST endpoint exposing a tenant-scoped resource by identifier.
Constraint adversarial test	A concurrent attempt to activate two leases against the same unit_id, issued from two simultaneous threads, results in exactly one successful insert and one unique constraint violation, verifying the partial index from Section 4.2 under real concurrency rather than in isolation.
Idempotency adversarial test	Triggering the invoicing batch job twice in succession for the same lease_id and billing_period results in exactly one row in public.invoices, verifying the composite unique index from Section 4.3 under a deliberately duplicated job execution.
Audit rollback adversarial test	A business transaction that publishes an AuditDomainEvent but subsequently rolls back results in zero rows written to system_audit_logs, verifying the AFTER_COMMIT semantics from Section 5.1 under a deliberately engineered failure.

TRD v2.1.0 — SYSTEM CORE ARCHITECTURAL BLUEPRINT

Multi-Tenant Property Management SaaS Platform · Java 21 / Spring Boot / PostgreSQL · June 2026